



JAVA WEB PROGRAMMING

THYMELEAF AND SPRING MVC INTEGRATION

Spring Framework 7 / Spring Boot 4 / Thymeleaf 3.1

Lecture Contents



SpringStandard Dialect



Spring Template Engine



Spring Boot Configuration for Thymeleaf



View and ViewResolver



Thymeleaf Components (radio, checkbox, select)



Rendering Thymeleaf Fragments



Async Technologies (AJAX, htmx, Unpoly, Turbo)

Introduction

Thymeleaf + Spring MVC integration enables:

- Redirecting @Controller methods to Thymeleaf templates
- Using Spring Expression Language (SpEL) expressions
- Creating forms fully integrated with beans and result bindings
- Displaying multilingual messages (i18n) via MessageSource
- Processing templates using Spring's mechanisms
- Automatic configuration via Spring Boot Starter Thymeleaf

Stack: Spring Framework 7.0 + Spring Boot 4 + Thymeleaf 3.1 + Java 17+

Introduction – Maven Dependencies

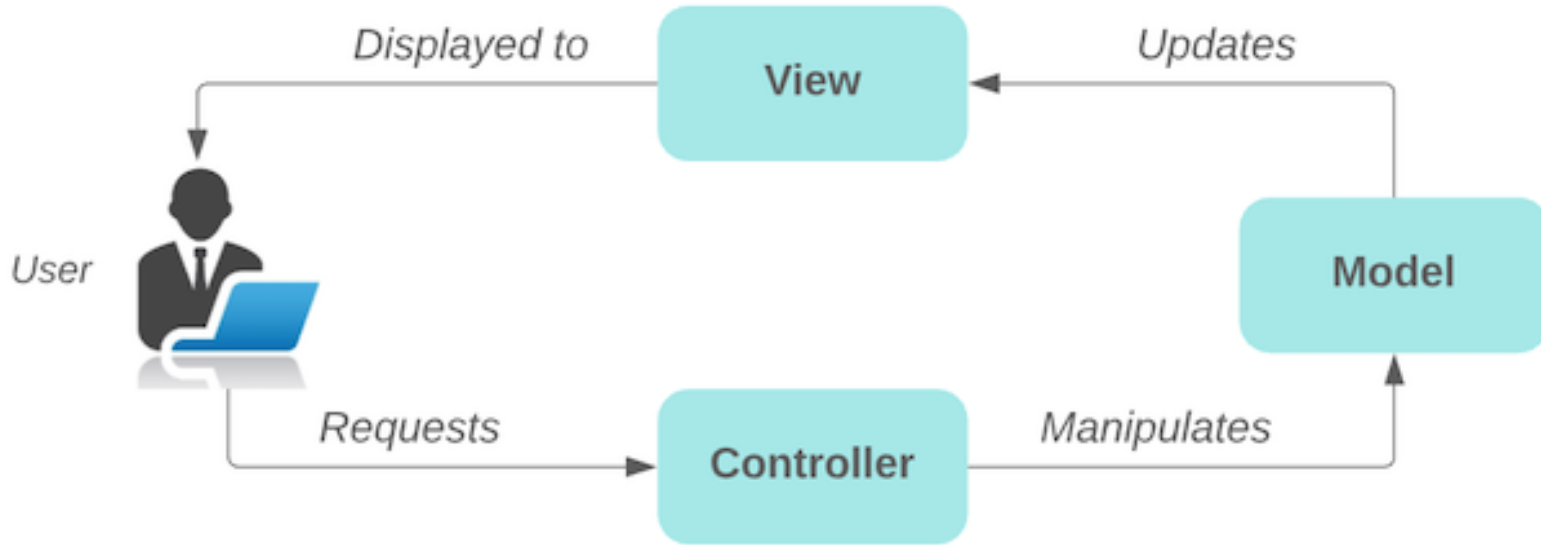
To integrate Thymeleaf and Spring MVC, add the following to pom.xml:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.4</version>
</parent>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

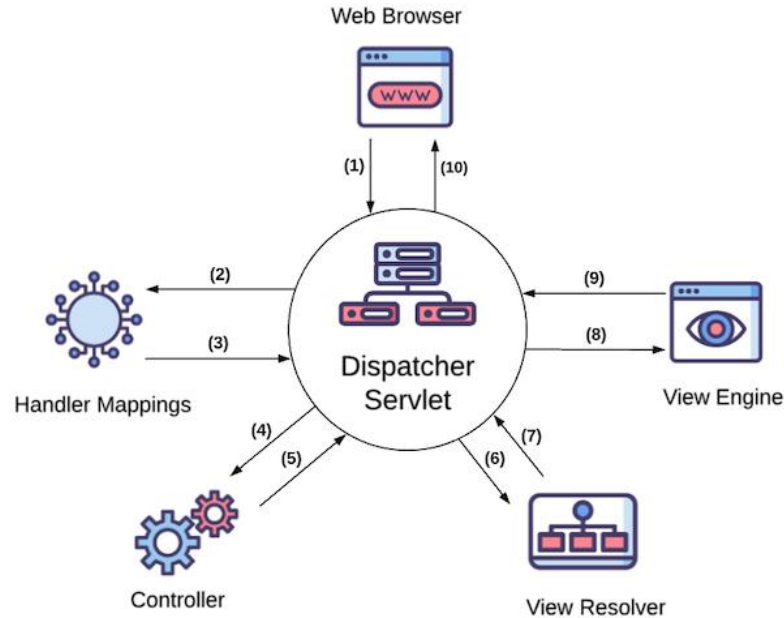
Spring Boot 4 auto-configures Thymeleaf – no manual configuration needed!

Introduction – MVC Architecture with Thymeleaf



Controller handles requests, Model holds data, View (Thymeleaf) renders HTML, and ViewResolver maps logical names to templates.

Introduction – Thymeleaf Template Engine



Thymeleaf Template Engine merges HTML templates with Model data using the SpringStandard Dialect and generates the final HTML output.

SpringStandard Dialect



- Implemented in `org.thymeleaf.spring6.dialect.SpringStandardDialect`
- Uses Spring Expression Language (SpEL) for `${...}` and `*{...}` expressions
- Bean access: `#{@beanName.method()}`
- Custom form attributes: `th:field`, `th:errors`, `th:errorclass`
- Equivalents for `th:object` enabling command object selection
- Utility expressions: `#themes.code(...)`, `#mvc.uri(...)`
- SpEL compiler support for faster execution (Spring 4.2.4+)

Note: Spring Framework 7 uses thymeleaf-spring6 library (Jakarta EE 9+)

Spring Template Engine



- Sets up configuration for Thymeleaf templates
- Defines prefix and suffix settings via TemplateResolver
- SpringTemplateEngine automatically applies SpringStandardDialect
- Enables Spring's MessageSource for internationalization
- SpEL compiler can be enabled for better performance
- In Spring Boot 4 – fully auto-configured!

Spring Template Resolver

```
@Bean
public SpringResourceTemplateResolver templateResolver() {
    SpringResourceTemplateResolver resolver =
        new SpringResourceTemplateResolver();
    resolver.setApplicationContext(this.applicationContext);
    resolver.setPrefix("classpath:/templates/");
    resolver.setSuffix(".html");
    resolver.setTemplateMode(TemplateMode.HTML);
    resolver.setCharacterEncoding("UTF-8");
    resolver.setCacheable(true);
    return resolver;
}
```

Manual configuration – only needed if you are NOT using Spring Boot auto-configuration.

Spring Template Engine – Configuration

```
@Bean
public SpringTemplateEngine templateEngine() {
    SpringTemplateEngine engine = new SpringTemplateEngine();
    engine.setTemplateResolver(templateResolver());
    // SpEL compiler speeds up execution
    engine.setEnableSpringELCompiler(true);
    return engine;
}
```

SpringTemplateEngine automatically activates SpringStandardDialect and Spring's MessageSource mechanism for localization.

Spring Boot Configuration for Thymeleaf



application.properties

```
# Thymeleaf settings (Spring Boot 4)
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.mode=HTML
spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.cache=true
# Set to false for development (hot reload)
# spring.thymeleaf.cache=false
```

application.yml (recommended for Spring Boot 4)

```
spring:
  thymeleaf:
    prefix: classpath:/templates/
    suffix: .html
    mode: HTML
    encoding: UTF-8
    cache: true
```

View and ViewResolver



The template system is based on two key interfaces:

View

- Responsible for rendering the HTML interface.
- Defines how data is displayed to the user.
- Can be customized for specific behavior.

ViewResolver

- Retrieves View objects for a specific operation.
- Maps logical names from Controller to View.
- Executes in an ordered chain.

Controller uses ViewResolver to redirect logic to the appropriate View template.

View and ViewResolver – konfiguracija

```
@Bean
public ThymeleafViewResolver viewResolver() {
    ThymeleafViewResolver viewResolver =
        new ThymeleafViewResolver();
    viewResolver.setTemplateEngine(templateEngine());
    viewResolver.setOrder(1);
    viewResolver.setCharacterEncoding("UTF-8");
    viewResolver.setViewNames(
        new String[] { "*.html", "*.xhtml" });
    return viewResolver;
}
```

In Spring Boot 4, ViewResolver is auto-configured – this code is only needed for custom configurations.

Thymeleaf Components – RadioButton



```
// Controller (Spring Framework 7)
@GetMapping("/film")
public String showForm(Model model) {
    model.addAttribute("film", new Film());
    model.addAttribute("sviZanrovi",
        List.of("Action", "Comedy", "Drama", "Horror"));
    return "film-form";
}

<!-- Thymeleaf template -->
<div th:each="zanr : ${sviZanrovi}">
    <input type="radio" th:field="*{zanr}"
        th:value="${zanr}" />
    <label th:for="${#ids.prev('zanr')}"
        th:text="${zanr}">Genre</label>
</div>
```

Select Movie Genre

☒ Action ----- th:field="*{zanr}"
type="radio"

☐ Comedy

☐ Drama

☐ Horror

Save

Wireframe: RadioButton Component

Thymeleaf Components – Checkbox



```
// Controller
@GetMapping("/smjestaj")
public String showFeatures(Model model) {
    model.addAttribute("smjestaj", new Smjestaj());
    model.addAttribute("sveZnacajke",
        List.of("Wi-Fi", "Parking", "AC", "Pool"));
    return "smjestaj-form";
}
```

```
<!-- Thymeleaf template -->
<ul>
    <li th:each="z : ${sveZnacajke}">
        <input type="checkbox"
            th:field="*{znacajke}"
            th:value="${z}" />
        <label th:for="${#ids.prev('znacajke')}"
            th:text="${z}">Feature</label>
    </li>
</ul>
```

Select Features

☒ Wi-Fi

☐ Parking

☒ AC

☐ Pool

----- th:field="*{znacajke}"
type="checkbox"

Save

Wireframe: Checkbox Component

Thymeleaf – Select (Dropdown)



```
// Controller
@GetMapping("/proizvod")
public String showProducts(Model model) {
    model.addAttribute("proizvod", new Proizvod());
    model.addAttribute("sveKategorije",
        List.of("Electronics", "Clothing", "Food"));
    return "proizvod-form";
}
```

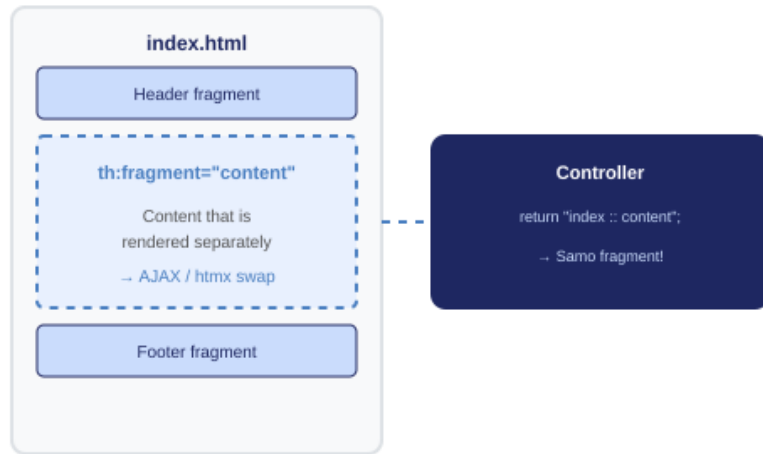
```
<!-- Thymeleaf template -->
<select th:field="*{kategorija}">
    <option value="">-- Select --</option>
    <option th:each="k : ${sveKategorije}"
        th:value="${k}"
        th:text="${k}">Category</option>
</select>
```

Wireframe: Select/Dropdown Component

Rendering Thymeleaf Fragments



- Thymeleaf allows rendering only part of a template (a fragment)
- Useful for AJAX / htmx calls – updating only part of the page
- Defining a fragment: `th:fragment="name"`
- Referencing a fragment: `"template :: name"`
- Referencing by ID: `"template :: #id"`
- Controller returns only the fragment, not the entire page



Wireframe: Fragment Structure in Template

Fragments – Specifying via ThymeleafView

```
@Bean(name = "content-part")
@Scope("prototype")
public ThymeleafView someViewBean() {
    ThymeleafView view = new ThymeleafView("index");
    view.setMarkupSelector("content");
    return view;
}
```

@Scope("prototype")

– returns a new instance for every request

setMarkupSelector("content")

– selects only the fragment named "content"

Fragments – Return Value from Controller

```
// Returning bean name (ThymeleafView)
@GetMapping("/showContentPart")
public String showContentPart() {
    return "content-part";
}

// Returning fragment by name
@GetMapping("/showContentPart")
public String showContentPart() {
    return "index :: content";
}

// Returning fragment by ID
@GetMapping("/showContentPart")
public String showContentPart() {
    return "index :: #content";
}

// Fragment with parameter
@GetMapping("/showContentPart")
public String showContentPart() {
    return "index :: #content ('myvalue')";
}
```

Fragments – Definition in HTML Template

Using the th:fragment attribute:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <header th:fragment="header">
    <nav>...</nav>
  </header>

  <div th:fragment="content">
    Only this part will be rendered!
  </div>

  <footer th:fragment="footer">
    <p>Footer content</p>
  </footer>
</body>
</html>
```

Controller: return "index :: content"; → renders only the <div> with fragment "content"

Fragments – Referencing by ID

```
// ThymeleafView bean s ID selektorem
@Bean(name = "content-part")
@Scope("prototype")
public ThymeleafView someViewBean() {
    ThymeleafView view = new ThymeleafView("index");
    view.setMarkupSelector("#content"); // CSS selektor!
    return view;
}
```

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <div id="content">
        Only this part will be rendered!
    </div>
</body>
```

Advantage: Using the #id selector does not require a th:fragment attribute in HTML.

Async Technologies – jQuery AJAX (Legacy)



```
<!-- HTML: select + div for replacement -->
<select id="selection">
  <option>Content 1</option>
  <option>Content 2</option>
</select>
<div id="replace_div"></div>

<!-- jQuery AJAX call -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js">
</script>
<script>
$(document).ready(function() {
  function getContent() {
    var url = "/content/";
    url += ($('#selection').val() === "Content 1")
      ? "content1" : "content2";
    $('#replace_div').load(url);
  }
  getContent();
  $('#selection').change(getContent);
});
</script>
```

⚠ jQuery approach requires manually writing JavaScript code for each interaction.

AJAX Fragments – Spring MVC Controller

```
@Controller
@RequestMapping("/content")
public class ContentController {

    @GetMapping
    public String loadContent() {
        return "website";
    }

    @GetMapping("/content1")
    public String getContent1() {
        return "content :: content1";
    }

    @GetMapping("/content2")
    public String getContent2() {
        return "content :: content2";
    }
}
```

Each method returns only the specified Thymeleaf fragment – not the entire page.

htmx – Modern Alternative to AJAX



```
<!-- Including htmx (~14kB) -->
<script src="https://unpkg.com/htmx.org@2.0.4"></script>

<!-- Declarative approach: no JavaScript needed! -->
<button hx-get="/content/content1"
        hx-target="#replace_div"
        hx-swap="innerHTML">
  Load Content 1
</button>

<select hx-get="/content"
        hx-target="#replace_div"
        hx-trigger="change"
        name="selection">
  <option value="content1">Content 1</option>
  <option value="content2">Content 2</option>
</select>
<div id="replace_div"></div>
```

htmx uses HTML attributes instead of JavaScript – a declarative, clean approach!

htmx + Spring Boot 4 Integration

Maven dependency:

```
<!-- htmx-spring-boot integracija -->
<dependency>
  <groupId>io.github.wimdeblauwe</groupId>
  <artifactId>htmx-spring-boot</artifactId>
  <version>4.0.1</version>
</dependency>
<dependency>
  <groupId>io.github.wimdeblauwe</groupId>
  <artifactId>htmx-spring-boot-thymeleaf</artifactId>
  <version>4.0.1</version>
</dependency>
```

```
// Controller with @HxRequest annotation
@Controller
public class ContentController {
    @HxRequest // htmx requests only
    @GetMapping("/content1")
    public String getContent1(HtmxRequest req) {
        return "content :: content1";
    }
}
```

htmx-spring-boot provides @HxRequest, HtmxResponse and Thymeleaf dialect for hx- attributes.*

Other Modern Alternatives – Unpoly and Turbo

Unpoly

- Uses [up-***] attributes in HTML
- Automatic fetching of page fragments only
- Layer system (modals, popovers)
- Navigation without refresh (up-follow)
- Form handling: up-submit
- Advanced page transitions

Hotwired Turbo

- Developed by Basecamp (Ruby on Rails)
- Turbo Drive: navigation without refresh
- Turbo Frames: replacing page sections
- Turbo Streams: server-push updates
- Well integrated with WebSockets
- Koristi `<turbo-frame>` custom elements

jQuery AJAX vs htmx – Visual Comparison

jQuery AJAX

```
$.ajax({ url: '/api',  
success: function(data) {  
  $('#div').html(data);  
}  
});
```

- † Requires JavaScript
- † jQuery dependency (~87kB)
- † Imperative style

htmx

```
<button  
  hx-get="/api"  
  hx-target="#div"  
  hx-swap="innerHTML">  
  Load  
</button>
```

- ✓ Declarative HTML attributes
- ✓ Only ~14kB
- ✓ No JavaScript code

htmx eliminates the need for JavaScript code and jQuery dependency – uses declarative HTML attributes.

Comparison of Async Technologies



Feature	jQuery AJAX	Fetch API	htmx	Unpoly	Turbo
Size	~87 kB	0 kB (native)	~14 kB	~42 kB	~25 kB
JavaScript	Yes (a lot)	Yes (a lot)	No (attributes)	No (attributes)	Minimal
Approach	Imperative	Imperative	Declarative	Declarative	Declarative
Thymeleaf Integration	Manual	Manual	Excellent	Good	Good
Spring Boot Support	None	None	htmx-spring-boot	None	None
SPA Feel	Partial	Partial	hx-boost	up-follow	Turbo Drive
State Complexity	Client-side	Client-side	Server-side	Server-side	Server-side
SEO	Problematic	Problematic	Excellent	Excellent	Excellent

Recommendation for 2026: htmx is the best choice for Spring Boot + Thymeleaf projects.

References



- <https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>
- <https://docs.spring.io/spring-framework/reference/web/webmvc-view/mvc-thymeleaf.html>
- <https://docs.spring.io/spring-boot/appendix/application-properties/index.html>
- <https://htmx.org/>
- <https://github.com/wimdeblauwe/htmx-spring-boot>
- <https://unpoly.com/>
- <https://turbo.hotwired.dev/>
- <https://spring.io/blog/2024/03/15/hypermedia-and-browser-enhancement/>
- <https://spring.io/guides/gs/serving-web-content/>



Thank You for Your Attention!

Spring Framework 7 / Spring Boot 4 / Thymeleaf 3.1

Academic Year 2025/2026